VISION MISSION OF THE ISNTITUTE

vision:

To be a globally recognized university through excellence in Education,

Innovation and Sustainable growth. Mission:

Deliver collaborative education to prepare students for global challenges through

1. Transformative learning
2. Vibrant research ecosystem
3. Sustainable and inclusive community

VISION MISSION OF THE DEPARTMENT

Vision:

To be a leading hub in Artificial Intelligence through excellence in education, innovation, and sustainable practices.

Mission:

M1: Deliver cutting-edge AI education with advanced tools, techniques, and industry collaboration to address global challenges.

M2: Nurture creativity, research, and lifelong learning for personal and professional excellence

M3: Promote ethical leadership and sustainability to build a responsible AI ecosystem

Program Educational Objectives (PEOs)

- **PEO 1 :** Apply emerging tools and technologies to solve interdisciplinary problems.
- **PEO 2 :** Advance in education and research by applying innovative ideas and project-based learning.
- **PEO 3 :** Work collaboratively and establish leadership with ethical, social, and financial accountability.

Program Specific Outcomes (PSOs)

- **PSO 1 :**Apply core engineering, computing principles, AI and programming skills to develop efficient software solutions.
- **PSO 2 :**Utilize emerging technologies to solve real-world challenges in industry and society with innovation and responsibility.

Index

S.NO	Table of Content	Page No.
1	**Introduction**	
	1.1 Introduction	1
	1.2 Problem Statement	1
	1.3 Objectives	1
	1.4 Scope of the Project	2
2	**System Overview**	
	2.1 Existing System	3
	2.2 Limitations of Existing System	3
	2.3 Proposed System	3
	2.4 Software Requirements	4
	2.5 Hardware Requirement	4
3	**Technologies Used**	
	3.1 Java	5
	3.2 python	5
	3.3 JDBC	5
	3.4 Database	6

4 Database design

4.1 Database Tables 7

4.2 Table Structure 7

4.3 Keys and Constraints 9

4.4 Table Relationships 10

5 System Modules

5.1 Add Record 11

5.2 Search Record 11

5.3 Update Record 11

5.4 Delete Record 12

6 Implementation

6.1 Database connection 13

6.2 Input Validation 13

6.3 Exception Handling 13

7 Output Screen

	7.1 Main menu screen	14
	7.2 user input	14
	7.3 Result Screen	15
	7.4 User profile	16
	7.5 Predicted _result screen	16
8	**Conclusion**	17
9	**Future Enhancement**	
	9.1 User Interface	18
	9.2 Real time data	18
	9.3 User security	18
10	**References**	19
11	**Appendix**	
	11.1 Java Code Snippets	20
	11.2 SQL Commands Used to Create Tables	22

No	Table of Pictures	Page No
1	Fig 7.1- Main menu screen	14
2	Fig 7.2- user input	14
3	Fig 7.3- result screen	15
4	Fig 7.4- user profile	16
5	Fig 7.5 – predicted_result screen	16

No	List of Tables	Page No

1	Table 4.1-User table structure	7
2	Table 4.2-prediction_results table structure	7

ABSTRACT

This project PLACEMENT PREDICTION TOOL presents a placement prediction and company-role recommendation system developed using a Java console application as the frontend and a Python-based backend. The system allows users to enter their academic details, skills, and desired job preferences. The Java application collects user input and sends it to the Python backend through API communication. The backend processes the data using prediction logic and returns placement probability along with suitable company and role suggestions. The system also evaluates the user's resume text and calculates a resume score.

The application uses JDBC connectivity to interact with a SQLite database. The database stores user details, prediction results, and history for future reference. This ensures efficient data management and retrieval. The modular architecture allows easy addition of new features without affecting existing components. The system provides a simple and user-friendly interface for students. It helps users understand their placement chances and improve their profiles. The integration of Java, Python, and database technologies makes the system efficient and scalable.

Based on the analysis, it identifies missing skills required for specific job roles. The application stores user inputs and results in a database for future reference. This system helps students understand their placement chances and improve their profiles. It provides a simple and efficient way to guide career decisions. Overall, the project demonstrates integration between Java and Python for building a real-world prediction system.

CHAPTER 1: INTRODUCTION

1.1 Introduction

This project is designed to predict student placement chances using a Java-based console application integrated with a Python backend. The system collects academic and skill data from users and processes it through backend logic. It provides placement probability and company-role recommendations. The system also analyzes resume text to evaluate skills. It helps students understand their strengths and weaknesses. The architecture separates frontend and backend for better performance. It acts as a career guidance tool.

1.2 Problem Statement

Students often do not know their placement chances or suitable career paths. They lack clarity about which companies and roles fit their profiles. There is no simple system that analyzes both academic data and skills together. Manual evaluation is time-consuming and inaccurate. Students also do not know what skills they are missing. Existing tools are either complex or not personalized. This project solves these problems using a Java-Python integrated system.

1.3 Objectives of the Project

- The system aims to recommend suitable companies and roles.
- It also evaluates resume text to generate a score.
- Another goal is to identify missing skills for improvement.
- The project stores user data and prediction results.
- It focuses on providing simple and fast predictions.
- The system provides a simple and user-friendly interface for students.

1.4 Scope of the Project

The system can be used by students for self-assessment. It helps in understanding placement readiness. The project can be extended to include real-time job data. It supports analysis of both academic and skill-based parameters. The system is limited to predefined logic and datasets. Future improvements can include web-based interfaces. It mainly focuses on prediction and recommendation.

CHAPTER 2: SYSTEM OVERVIEW

2.1 Existing System

Existing systems provide general placement guidance without personalization. They do not combine academic and skill-based analysis effectively. Most systems lack backend intelligence for prediction. There is no proper integration between frontend and backend technologies. Resume evaluation is not included. Data is not stored efficiently for future use. Hence, existing systems are limited in functionality.

2.2 Limitations of Existing System

Existing systems do not provide accurate predictions. They lack integration with backend processing systems. Company-role matching is not available. Resume analysis is missing. Data storage and history tracking are not implemented. Systems are not user-friendly or interactive. They fail to provide complete career guidance.

2.3 Proposed System

The proposed system uses a Java console application for user interaction. It connects to a Python backend for processing data. The system predicts placement probability based on input data. It suggests companies and roles suitable for the user. It analyzes resume text and provides a score. It identifies missing skills for improvement. It stores results in a database for future reference. The system provides a centralized platform for food donors and receivers.

2.4 Software Requirements

- Operating System: Windows / Linux
- Programming Language: Java (JDK 8 or above)
- IDE: VS Code
- Database: MySQL
- GitHub (for version control)

2.5 Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 20 GB
- Input Devices: Keyboard, Mouse
- Output Device: Monitor

CHAPTER 3: TECHNOLOGIES USED

3.1 Java

Java is used to develop the console-based frontend of the system. It provides a user interface where users can enter their academic details, skills, and job preferences. The application uses a menu-driven approach to perform operations like add, search, update, and delete records. Java collects user input and converts it into a structured format. It then sends this data to the Python backend using API calls. After processing, the response from the backend is received and displayed to the user. Java also handles input validation to ensure correct data entry. It controls the overall flow of the application. The frontend ensures smooth interaction between the user and the system. It acts as the communication layer between the user and backend services.

3.2 Python

Python is used to implement the backend logic of the system. It receives user data from the Java frontend through API requests. The backend processes this data to calculate placement probability. It also performs company-role matching based on user input. Resume text is analyzed to generate a resume score and identify missing skills. Python uses libraries and logic to perform efficient data processing. Flask is used to create APIs for communication. The backend returns structured results to the Java application. It ensures fast and accurate predictions. Python acts as the core engine of the system where all computations take place.

3.3 JDBC

JDBC (Java Database Connectivity) is a standard Java API that acts as a bridge between the Java application and the MySQL database to execute SQL queries. It is used to establish a secure connection, allowing the system to perform essential CRUD operations such as storing donor information and retrieving available food records. By utilizing the

DriverManager and PreparedStatement classes, JDBC ensures that data is handled efficiently while maintaining high levels of data integrity and security.

3.4 Database

SQLite is used as the database in this project to store user data and prediction results. It is a lightweight, file-based database that does not require a separate server. The database stores information such as academic details, skills, resume text, and placement predictions. It is integrated with the Java frontend using JDBC connectivity. SQLite allows efficient execution of CRUD operations like insert, update, delete, and retrieve. The database also maintains history of user records for future reference. It ensures data consistency and quick access to stored information. Since it is easy to set up, it is suitable for small to medium-scale applications. SQLite improves the overall performance of the system by providing fast data handling. It plays an important role in maintaining persistent storage in the project.

CHAPTER 4: DATABASE DESIGN

4.1 Database tables

The project utilizes two primary tables to manage user data and prediction results:

- **User_Details Table:** Stores user information including academic details, skills, and resume data entered through the Java frontend.
- **Prediction_Results Table:** Stores the output generated by the Python backend such as placement probability, recommended company, role, resume score, and missing skills.

4.2 Table Structure

User table:

Prediction_results table:

Field	Data Type	Description
user_id	int	A unique identifier assigned to each user, serving as the Primary Key.
name	varchar(100)	Stores the name of the user.
gender	varchar(10)	Stores the gender of the user.
ssc_p	real	Stores 10th percentage of the user.

Field	Data Type	Description
hsc_p	**real**	**Stores 12th percentage of the user.**
degree_p	**real**	**Stores degree percentage of the user.**
mba_p	**real**	**Stores MBA percentage of the user.**
workex	**varchar(10)**	**Indicates whether the user has work experience (Yes/No).**
skills	**text**	**Stores the list of skills entered by the user.**
resume_text	**text**	**Stores the resume content provided by the user.**

Table 4.1-User table structure

Prediction_results table:

Field	Data Type	Description
result_id	int	A unique identifier for each prediction record, used as the Primary Key.
user_id	int	Foreign key linking to the User_Details table.

Field	Data Type	Description
placement_probability	real	Stores the predicted placement probability.
company	varchar(100)	Stores the suggested company name.
role	varchar(100)	Stores the recommended job role.
resume_score	real	Stores the calculated resume score.
missing_skills	Text	Stores the list of skills the user needs to improve.

Table 4.2-donations table structure

4.3 Keys and Constraints

To maintain data integrity and ensure system reliability, the following constraints are applied:

- **Primary Key (PRI):**
  The user_id in the User_Details table and result_id in the Prediction_Results table act as primary keys. These uniquely identify each record in the database.

- **Auto-Increment:**
  Both tables use auto-increment for primary key fields to automatically generate unique IDs for each new record.

- **Foreign Key (FK):**
  The user_id in the Prediction_Results table acts as a foreign key referencing the User_Details table. This ensures that each prediction result is linked to a valid user.

- **NOT NULL Constraint:**
  Important fields such as user_id, placement_probability, and company are restricted from storing null values.

These constraints ensure data consistency, avoid duplication, and maintain referential integrity.

4.4 Tables and relationships:

The database is designed using a relational model where the **User_Details table acts as the parent entity** and the **Prediction_Results table acts as the child entity**.

This creates a **One-to-Many (1:N) relationship**, as a single user can have multiple prediction results over time.

The relationship is established using the user_id field, which connects both tables. This allows the system to track all predictions made for a particular user.

This structure ensures that each prediction result is associated with a valid user profile. It also enables efficient retrieval of user history and improves data organization.

CHAPTER 5: SYSTEM MODULES

Module 1: Add Record (User data entry)

The Add Record module allows users to enter their academic details, skills, and resume information into the system. The Java console application collects inputs such as name, scores, work experience, and skills from the user. This data is then sent to the Python backend through API communication for processing. Simultaneously, using JDBC (Java Database Connectivity), the system executes an **INSERT SQL command** to store the user details in the SQLite database. The backend processes the input and generates prediction results. These results are also stored in the database for future reference. This module ensures that all user information is properly captured and saved.

Module 2: Search Record (results retrival)

The Search Record module enables users to retrieve stored data and prediction results from the database. Users can search records using a unique user ID or name. When a search request is made, the system performs a **SELECT SQL operation** using JDBC to fetch relevant records from both user_details and prediction_results tables. The retrieved data includes academic details, placement probability, company suggestions, and roles. The results are displayed in the Java console interface. This module helps users review their previous predictions. It provides quick and efficient access to stored information.

Module 3: Update Record (profile improvement)

The Update Record module allows users to modify their existing details such as academic scores, skills, or resume text. When the user updates their data, the system identifies the record using the primary key (user_id). It then executes an **UPDATE SQL command** through JDBC to modify the database entry. After updating, the modified data is sent again to the Python backend for reprocessing. The system generates updated placement predictions and recommendations. This ensures that the results reflect the latest user profile. The module helps users improve their data and get better insights.

Module 4: Delete Record (Data Management)

The Delete Record module is responsible for removing unwanted or outdated records from the system. If a user chooses to delete their data, the system identifies the specific record using the user ID. It then executes a **DELETE SQL command** through JDBC to remove the data from both user_details and prediction_results tables. This helps in maintaining clean and relevant data in the database. It prevents unnecessary storage usage and improves system performance. The module ensures proper data management and system maintenance.

CHAPTER 6: IMPLEMENTATION

6.1 Database Connection

The system uses **JDBC** to establish a connection between the Java console application and the SQLite database. This is achieved by loading the SQLite JDBC driver and connecting through a database URL. The application uses Connection and PreparedStatement objects to execute SQL queries such as INSERT, SELECT, UPDATE, and DELETE. The Java frontend handles all database interactions, ensuring smooth data flow. This connection enables persistent storage of user profiles and prediction history. It ensures that data remains even after the application is closed. Overall, it provides reliable and efficient data management.

6.2 Input Validation

To ensure data accuracy and integrity, the system performs input validation before processing any user data. The Java application checks that all required fields such as name, academic scores, and skills are properly entered. It validates numerical inputs like percentages to ensure they are within valid ranges. The system prevents empty or incorrect inputs from being submitted. It also ensures that fields like work experience accept only valid values (Yes/No). Before storing data, the system checks for duplicate entries where necessary. It ensures that only valid data is sent to the Python backend for processing.

6.3 Exception Handling

The system includes exception handling mechanisms to manage runtime errors effectively. The Java application uses try-catch blocks to handle exceptions such as SQLException and input-related errors. If a database connection fails or a query is incorrect, the system catches the error and displays a user-friendly message.. This prevents the application from crashing unexpectedly. Exception handling ensures smooth execution of the program even under error conditions. Overall, it enhances user experience by providing clear error messages and safe recovery.

CHAPTER 7: OUTPUT SCREENS

7.1 Main Menu Screen:

The Main Menu screen is the entry point of the application where users interact with the system through a Java console interface. It displays options such as Analyze (Modes A/B/C) and save history (JDBC/SQLite), View History, and Exit. This screen allows users to navigate between different modules easily. It ensures smooth interaction between the user and the system. The menu-driven approach makes the application simple and user-friendly. It acts as the control panel for accessing all functionalities.

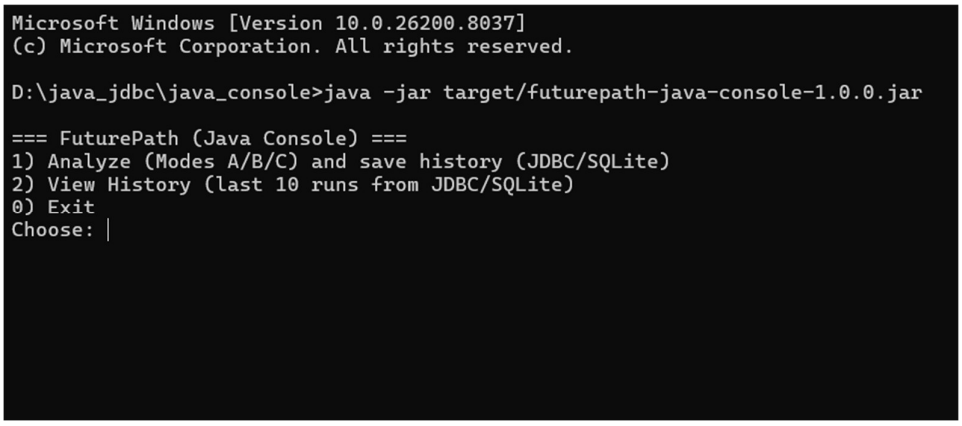

Fig 7.1- Main Menu screen

7.2 User Input:

The User Input screen allows users to enter their academic details, skills, and resume information. It collects data such as name, scores, work experience, and skill set. The input is validated before being processed. Once submitted, the data is sent to the Python backend for prediction. This screen is essential for capturing user information accurately. It ensures that all required details are entered correctly. It acts as the starting point for prediction analysis.

```
D:\java_jdbc\java_console>java -jar target/futurepath-java-console-1.0.0.jar

=== FuturePath (Java Console) ===
1) Analyze (Modes A/B/C) and save history (JDBC/SQLite)
2) View History (last 10 runs from JDBC/SQLite)
0) Exit
Choose: 1
Gender (M/F): M
SSC %: 50
HSC %: 50
HSC Stream (Science/Commerce/Arts): science
Degree %: 50
Degree Type (Sci&Tech/Comm&Mgmt/Others): sci
WorkEx (Yes/No): no
E-test %: 60
Specialisation (Mkt&Fin/Mkt&HR): mkt
MBA %: 50
Resume PDF path (empty to paste text): "C:\Users\Bharath\Downloads\Bakka_Sri_
Could not read PDF text. Paste resume text:

Target Company (optional):
Target Role (optional): ai engineer
```

Fig 7.2- User Input Screen

7.3 Result Screen:

The Prediction Result screen displays the output generated by the Python backend. It shows placement probability, suggested company, job role, and resume score. It also highlights missing skills required for improvement. The results are displayed clearly in the console for user understanding. This screen helps users analyze their placement chances. It provides meaningful insights based on input data. It acts as the core output of the system.

```
--- RESULT ---
Mode: C
Placement Probability: 35.29%
Top Matches:
 - {ats=15, company=Notable, fit_score=19, role=Senior ML / AI Engineer, role_fit=15}
 - {ats=15, company=Eleven Recruiting, fit_score=19, role=ML AI Engineer, role_fit=15}
 - {ats=15, company=The Judge Group, fit_score=19, role=Manufacturing Data Process (Controls / AI) Engineer, role_fit=15}
 - {ats=15, company=Foundation Talent, fit_score=19, role=AI Engineer (Machine Learning / LLM'S) Well funded Startup backed by the best, role_fit=15}
 - {ats=15, company=CyberCoders, fit_score=19, role=Machine Learning AI Engineer/Scientist, role_fit=15}
 - {ats=15, company=Keylent Inc, fit_score=19, role=Generative AI Engineer/Sr. Data Scientist, role_fit=15}
 - {ats=15, company=Capgemini Engineering, fit_score=19, role=Senior Data & AI Engineer - Senior Data Scientist, role_fit=15}
 - {ats=15, company=Dice, fit_score=19, role=Lead Data Engineer, role_fit=15}
 - {ats=15, company=University of Chicago, fit_score=19, role=Senior Data Engineer, role_fit=15}
 - {ats=15, company=Experfy, fit_score=19, role=Sr. Data Engineer, role_fit=15}
```

Fig 7.3- Result screen

7.4 user profile:

The Donor Profile screen retrieves and displays specific user information from the database, such as the donor's name and email. This allows users to verify their registration details and manage their active presence within the system.

Fig 7.4- user profile

7.5 Prediction history screen:

It shows placement probability, company suggestions, and roles for each record. This helps users track their progress over time. It provides a clear record of all past analyses.

Fig 7.5- history screen

CHAPTER 8: CONCLUSION

The PLACEMNT PREDICTION TOOL successfully demonstrates how a Java-based frontend can be integrated with a Python backend to build a placement prediction and recommendation application. The system collects user academic details, skills, and resume data through a console interface. It processes this information efficiently and generates meaningful outputs such as placement probability and suitable job roles. The modular architecture ensures smooth communication between components. Overall, the system provides a practical solution for career guidance.

The implementation of the Python backend plays a key role in analyzing user input and generating accurate predictions. It processes the data received from the Java application and returns structured results. The system also performs resume evaluation and identifies missing skills required for improvement. This helps users understand their strengths and weaknesses. The backend ensures fast and efficient processing of data.

The use of JDBC connectivity with a SQLite database ensures proper storage and management of data. User details and prediction results are stored securely and can be retrieved when needed. The database supports CRUD operations, allowing users to add, update, search, and delete records. This improves system reliability and usability. It also helps maintain history for future analysis.

In conclusion, the project provides an effective platform for predicting placement chances and guiding users toward better career decisions. It combines multiple technologies in a simple and efficient way. The system can be further enhanced by adding advanced features and real-time data integration. It demonstrates the importance of combining frontend, backend, and database systems. This project serves as a strong foundation for building intelligent career guidance applications.

CHAPTER 9: FUTURE ENHANCEMENTS

9.1 User Interface

The current system uses a Java console interface, which can be upgraded to a graphical or web-based interface. This will make the system more interactive and user-friendly. Users will be able to input data easily and view results in a structured format. A GUI will improve the overall user experience. It will also make the system more suitable for real-world usage.

9.2 Real -time data

The system can be enhanced by integrating real-time job data from online job portals. This will allow users to get updated company and role recommendations. The suggestions will be based on current industry requirements. It will make the system more dynamic and practical. This feature will help users align their skills with market demands.

9.3 User security

The system can include user login and authentication features. Users can create accounts and securely access their data. Role-based access can be implemented for better control. This will improve data security and privacy. It will also help in managing user-specific dataefficiently.

CHAPTER 10: REFERENCES

Books and Technical Manuals:

- “Java: The Complete Reference" by Herbert Schildt: This served as the primary guide for understanding Core Java concepts, multithreading, and the Collections Framework used in the project's backend.
- "Head First Servlets and JSP" by Bryan Basham: This provided essential insights into the Request-Response lifecycle.

Websites and Online Documentation:

- Oracle Java Documentation: Used for official API references regarding JDBC (Java Database Connectivity) and SQL exception handling.
- W3Schools & Tutorialspoint: These platforms were utilized for front-end styling with HTML/CSS and for structuring SQL queries to manage the "Add," "Search," and "Delete" record modules.
- MySQL Official Manual: Consulted for database schema design and establishing secure connections between the Java application and the local server.

Chapter 11: APPENDIX

11.1 JAVA CODE SNIPPETS:

11.1.1 DataBase Connection:

```
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package com.futurepath.db;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public final class Db {
   private Db() {
   }

   public static Connection connect() {
      try {
         Connection c = DriverManager.getConnection("jdbc:sqlite:futurepath.db");
         init(c);
         return c;
      } catch (Exception e) {
         throw new RuntimeException(e);
      }
   }

   private static void init(Connection c) {
      try {
         Throwable var1 = null;
         Object var2 = null;

         try {
            Statement st = c.createStatement();

            try {
               st.executeUpdate("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY AUTOINCREMENT,created_at TEXT NOT NULL,request_json TEXT NOT NULL,response_json TEXT NOT NULL)");
```

```
43              } finally {
44                 if (st != null) {
45                    st.close();
46                 }
47
48              }
49
50           } catch (Throwable var11) {
51              if (var1 == null) {
52                 var1 = var11;
53              } else if (var1 != var11) {
54                 var1.addSuppressed(var11);
55              }
56
57              throw var1;
58           }
59        } catch (Exception e) {
60           throw new RuntimeException(e);
61        }
62     }
63  }
64
```

64.1 SQL Commands Used to Create Tables:

64.1.1 Create Users Table:

```
CREATE TABLE user_details (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    gender TEXT,
    ssc_p REAL,
    hsc_p REAL,
    degree_p REAL,
    mba_p REAL,
    workex TEXT,
    skills TEXT,
    resume_text TEXT
);
```

64.1.2 Create prediction_result Table:

```
CREATE TABLE prediction_results (
result_id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER,
```

```
    placement_probability REAL,
    company TEXT,
    role TEXT,
    resume_score REAL,
    missing_skills TEXT,
    FOREIGN KEY (user_id) REFERENCES user_details(user_id)
);
```